

1. `corrected_de_analysis.py` (413 lines)

What it does

Master pipeline for rat hippocampal proteomics differential expression analysis. Steps:

1. Loads a DIA-NN output matrix (`diann_matrix.tsv`) and builds sample metadata (group assignments: CTRL, HFD, HFD_Pio, HFD_IP, HFD_Cr; wave assignments: Wave1/Wave2).
2. Removes contaminant proteins by keyword matching ("keratin", "trypsin", "serum albumin") in the Description column.
3. Log2-transforms intensity data (replacing 0 with NaN first).
4. Calls out to `run_limma_loess.R` via subprocess for cyclic loess normalization.
5. KNN imputation (k=5) using sklearn.
6. PCA (5 components) on the imputed, z-scaled data.
7. Calls out to `run_limma_ebayes.R` via subprocess for limma eBayes differential expression (4 contrasts: HFD vs CTRL, HFD_Pio vs HFD, HFD_IP vs HFD, HFD_Cr vs HFD).
8. Volcano plots for all 4 contrasts.
9. Z-scored clustermap heatmap of HFD-signature proteins.
10. Summary table.

Key parameters/thresholds

- **KNN k=5** (line 170)
- **Significance: FDR < 0.05 AND |log2FC| ≥ 1.0** (lines 262-263)
- **Contaminant keywords:** keratin, trypsin, serum albumin (line 93)
- **Model:** `~ 0 + group + wave` (cell-means with wave covariate, lines 226-230)
- **PCA:** 5 components, StandardScaler (lines 191-192)
- **Heatmap Z-score clipping:** vmin=-3, vmax=3 (line 370)

Potential issues

- **Line 46-47 – Wave assignment logic may be fragile/incorrect for treatment group samples.** The `wave1_prefixes` and `wave2_prefixes` sets only contain **AC** and **AHFD** prefixes. For treatment samples (e.g., those containing "Pio", "IP", "Cr"), the prefix extraction at lines 65-70 will produce things like "AHFD" + character-at-index-4, which may not match either wave set, so all treatment samples may default to **Wave2**. This could misassign wave for treatment group animals.
- **Line 66 – `prefix = 'AC' + name[2]`:** For a sample like "AC10", this takes `name[2]` which is "1", giving prefix "AC1" — this works for single-digit IDs but would give the wrong result for multi-digit IDs if they existed.
- **Line 68 – `prefix = 'AHFD' + name[4]`:** Same single-character extraction issue. For treatment group samples (e.g., "AHFD_Pio1"), `name[4]` would be `_` or **P**, producing a nonsense prefix like "AHFDP" or "AHFD_", which won't match either wave set.
- **Line 134 – `subprocess.run` uses `check=True`:** If the R script fails, the Python script will crash with a `CalledProcessError`. This is correct behavior but provides no custom error message.
- **Line 272 – `os.remove(OUTPUT_DIR / 'tables' / ...)` uses a Path object with `os.remove`:** This works in modern Python (3.6+) but is slightly unusual; `Path.unlink()` would be more idiomatic.
- **Lines 132-139 – Temp file cleanup:** If the R subprocess fails (line 135), the temp files `temp_log2.csv` and `temp_norm.csv` won't be cleaned up (no try/finally).

- **Line 198 – `meta['group'].values`:** The PCA result DataFrame assumes `meta` index order matches `imputed_df.columns` order. This is likely true since `meta` was built from `df.columns`, but after contaminant removal (which only removes rows), the column order should be preserved. However, there's no explicit alignment check.

2. `run_limma_loess.R` (18 lines)

What it does

Simple R helper script called by the main Python pipeline. Reads a CSV of log2-transformed data, runs `limma::normalizeCyclicLoess()` with `method="fast"`, and writes the result back to CSV.

Key parameters/thresholds

- **`method="fast"`** (line 13): Uses the fast approximation for cyclic loess, which handles NAs better than the default "affy" method.

Potential issues

- **Line 9 – `mat = as.matrix(df)`:** If the CSV contains NAs (which it will, since zeros were converted to NaN before log2), `normalizeCyclicLoess` with `method="fast"` should handle them, but the `"pairs"` method would not. The choice of `"fast"` is deliberate and documented, so this is fine.
- **No error handling:** If the input file doesn't exist or has wrong format, the script will crash with a generic R error.
- **Line 16 – `write.csv(norm_mat, outfile)`:** The output will have an `X` column as row names. The Python side reads with `index_col=0` (line 136 of main script), so this is consistent.

3. `run_limma_ebayes.R` (38 lines)

What it does

R helper that performs the limma eBayes differential expression testing. Reads the imputed expression matrix and a design matrix CSV, fits the linear model with `lmFit`, builds a contrast matrix for the 4 contrasts, runs `contrasts.fit + eBayes`, and saves `topTable` results (all genes, unsorted) for each contrast.

Key parameters/thresholds

- **Contrasts** (lines 20-24): `HFD - CTRL`, `HFD_Pio - HFD`, `HFD_IP - HFD`, `HFD_Cr - HFD`
- **`topTable(number=Inf, sort.by="none")`** (line 33): Returns all proteins, unsorted.
- **`eBayes()`** (line 29): Uses default prior settings (empirical Bayes moderation).

Potential issues

- **Line 13 – `design = as.matrix(design_df[colnames(df), ])`:** This indexes the design matrix rows by column names of the expression matrix. If column names in the expression data don't exactly match the row names in the design CSV (e.g., `.raw` suffix mangling by R's `make.names`), this will silently produce NAs or fail. R converts dots and hyphens in column names, which could cause a mismatch.

- **Line 16 – `lmFit(df, design)`:** `df` is a data.frame, not an explicit matrix. `lmFit` can handle both, but converting to matrix first (`as.matrix(df)`) would be safer.
- **No check that all expected contrast columns exist in the design matrix.** If the design matrix is missing a group (e.g., if one group had all samples filtered out), `makeContrasts` would fail.

4. `generate_qc_figures.py` (151 lines)

What it does

Post-hoc QC visualization script. Generates three figure panels:

1. **P-value histograms** (1×4 grid): Density histograms of raw p-values for each contrast with a uniform null reference line.
2. **MA plots** (2×2 grid): Average expression vs. log2FC with a LOWESS trend line, colored by significance.
3. **Sample correlation heatmap**: Pearson correlation clustered heatmap with group and wave color annotations.

Key parameters/thresholds

- **P-value histogram bins:** 40 (line 31)
- **MA plot LOWESS frac:** 0.3 (line 77)
- **Significance for MA coloring:** $FDR < 0.05$ AND $|\log_2FC| \geq 1.0$ (line 63)
- **Correlation heatmap:** Pearson, viridis colormap, vmin = matrix minimum (line 143)

Potential issues

- **Line 76 – `from statsmodels.nonparametric.smoothers_lowess import lowess`:** This import is inside the loop body. While Python caches imports, it's inefficient style and should be at the top of the file.
- **Line 59 – Fallback for missing AveExpr:** `df.mean(axis=1)` computes the mean across all columns of the DE results CSV, which includes non-numeric columns like "Protein", "Description", "significant_FDR05", etc. This would produce NaN/errors. The fallback should filter to numeric columns or, better, this condition should never be hit since limma's `topTable` always includes `AveExpr`.
- **Line 24 – `axes_flat = axes.flatten()`:** When `plt.subplots(1, 4, ...)` is called, the axes object is already 1D (shape (4,)), so `.flatten()` works but is redundant. Not a bug.
- **Lines 101-116 – Duplicated metadata construction logic:** The same metadata extraction code is copy-pasted from the main script. This is a maintenance risk — if sample naming conventions change, both files must be updated.

5. `downstream_analysis.R` (262 lines)

What it does

Comprehensive downstream/functional analysis of DE results. Steps:

1. **ID Mapping:** UniProt → Entrez ID and gene Symbol using `org.Rn.eg.db`, with fallback extraction of gene names from the Description column (`GN=` field).
2. **Rescue Analysis:** For each HFD-significant protein, computes % rescue by each intervention (Pio, IP, Cr) as $-(FC_{drug} / FC_{hfd}) \times 100$. Generates scatter and bar plots.

3. **GO and Reactome ORA:** Over-representation analysis of HFD-significant proteins (Entrez IDs) using **enrichGO** (BP) and **enrichPathway** (rat).
4. **UpSet Plot:** Overlap of significant proteins across all 4 contrasts (FDR < 0.05 only, no FC filter).
5. **GSEA:** Gene Set Enrichment Analysis on t-statistic-ranked genes using GO BP terms.
6. **STRING PPI Network:** Maps significant proteins to STRING v12.0 (rat, species 10116), generates network plot and enrichment table.
7. **Intervention Heatmap:** Z-scored heatmap of group means for the union of all significant proteins across contrasts, using ComplexHeatmap.
8. **Trajectory Plots:** Line plots of mean $\pm$ SE across groups for core HFD signature proteins ($|\log_2\text{FC}| \geq 1.0$).

Key parameters/thresholds

- **ORA significance:** FDR < 0.05 for both sig gene list and the **pvalueCutoff** (lines 30, 106, 114)
- **UpSet:** FDR < 0.05 only (no FC filter) (lines 127-130)
- **GSEA pvalueCutoff:** 1.0 (line 148) — intentionally permissive to capture all results
- **STRING:** version 12.0, species 10116 (rat), score_threshold 400 (line 158)
- **Rescue %:** $-(\text{FC}_{\text{drug}} / \text{FC}_{\text{hfd}}) \times 100$ (lines 64-66)
- **Trajectory filter:** **significant_FC1 == "True"** (line 223)

Potential issues

- **Line 223 – `filter(significant_FC1 == "True")`:** The column **significant_FC1** was written from Python as a boolean (**True/False**), but when read into R from CSV it becomes a character string **"True"** or **"FALSE"**. The comparison **== "True"** works, but is fragile. If Python writes **TRUE** (uppercase) or **1**, this will silently return zero rows. A more robust approach would be **as.logical()**.
- **Line 30 – `sig_hfd <- de_hfd %>% filter(adj_p_value < 0.05)`:** This uses FDR < 0.05 WITHOUT a fold-change filter for ORA and rescue analysis. This is a deliberate choice (cast a wider net for enrichment), but it's inconsistent with the **"significant_FC1"** filter used in trajectory plots and the main script's definition.
- **Line 134 – `UpSet sets.bar.color`:** The color vector has 4 elements, but the order depends on UpSetR's internal set ordering. If the sets are reordered alphabetically or by frequency, the colors may not match the intended groups.
- **Line 148 – `gseGO(pvalueCutoff = 1)`:** Setting the p-value cutoff to 1 means ALL gene sets are returned regardless of significance. This could produce very large output files and slow down execution. While this is presumably intentional (to see the full distribution), it's worth noting.
- **Line 159 – `string_db$map(sig_hfd, "Protein", removeUnmappedRows=TRUE)`:** The **sig_hfd** data frame has been augmented with **EntrezID** and **Symbol** columns via **left_join** on line 51. If there are duplicate rows due to multiple Entrez ID mappings, this could cause issues.
- **Line 252 – `scale_color_manual(values = col_colors)`:** The variable **col_colors** is defined much earlier in section 7 (line 206) inside a conditional block (**if (length(union_sig) > 0)**). If that block doesn't execute (no significant proteins), **col_colors** won't exist when the trajectory plot code runs, causing an error. **This is a real bug** — the variable scope depends on the conditional block executing first.

- **Line 258 – `height = max(4, ceiling(length(core_hfd)/3)*3)`:** The height formula could produce very large figures if many proteins pass the threshold (e.g., 100 proteins → height = 102 inches).

6. `plot_trajectories.R` (54 lines)

What it does

Standalone version of the trajectory plotting (Analysis 8 from `downstream_analysis.R`). Loads imputed data, maps UniProt IDs to gene symbols, filters for core HFD signature proteins ($|\log_2\text{FC}| \geq 1.0$), reshapes to long format, computes group means $\pm$ SE, and generates faceted line plots showing protein intensity trajectories across CTRL → HFD → interventions.

Key parameters/thresholds

- **Filter:** `significant_FC1 == "True"` (line 14) — same issue as `downstream_analysis.R`
- **Group factor levels:** CTRL, HFD, HFD_IP, HFD_Pio, HFD_Cr (line 27)
- **Facet layout:** 3 columns (line 45)
- **Plot dimensions:** width=10, height dynamically scaled (line 50)

Potential issues

- **Line 14 – `filter(significant_FC1 == "True")`:** Same fragility issue as `downstream_analysis.R` line 223. Python booleans written to CSV can be `True`, `TRUE`, `true`, or `1` depending on the pandas version and platform.
- **Line 11 – `mapIds` with no fallback:** Unlike `downstream_analysis.R`, this script does NOT include the fallback logic that extracts gene symbols from the `GN=` field in the Description column. So any proteins that fail to map via `org.Rn.eg.db` will have `NA` symbols, producing labels like "Q9JK11 (NA)" in the plot.
- **Line 31 – `map_df$Symbol[match(...)]`:** This works correctly but will produce NA for any protein in `summ_df` that isn't in `map_df`. Since `map_df` is built from `de_hfd$Protein` and `core_hfd` is a subset thereof, this should be fine.
- **Line 50 – `height = max(4, ceiling(length(core_hfd)/3)*3)`:** Same oversized-figure risk as `downstream_analysis.R` line 258.
- **Duplicated code:** This is essentially a copy of the trajectory section from `downstream_analysis.R` (lines 222-258). If one is updated, the other could fall out of sync.

Cross-cutting issues across the project

1. **Wave assignment for treatment groups (`corrected_de_analysis.py` lines 46-72, `generate_qc_figures.py` lines 101-116):** The prefix-based wave logic only handles AC/AHFD prefixes correctly. Treatment samples (Pio, IP, Cr) go through the `AHFD` branch but the character extraction at position 4 may grab the wrong character, likely defaulting them all to Wave2.
2. **Duplicated metadata logic:** The group-assignment logic (grepl-based) is duplicated in at least 4 files (`corrected_de_analysis.py`, `generate_qc_figures.py`, `downstream_analysis.R`, `plot_trajectories.R`). This should be centralized.
3. **Boolean column serialization (Python→R):** The `significant_FC1` column is a Python boolean written by pandas as `True/False`. R reads this as a character string. The R scripts

compare against `"True"` which happens to work for pandas' default serialization, but this is fragile and would break if `to_csv` settings changed.

4. **Temp file management:** The main script creates temp files (`temp_log2.csv`, `temp_norm.csv`, `temp_design.csv`) without try/finally cleanup. If an R subprocess fails, these files will remain on disk.
5. **No explicit column alignment checks** between the expression matrix and design matrix when passing data between Python and R via CSV.